



CS605PC - ARTIFICIAL INTELLIGENCE LAB MANUAL

(Based Strictly on AIMA Algorithms – Python)

Instructor: Shafakhatullah Khan Mohammed

List of Experiments

1. Breadth First Search
2. Depth First Search
3. Tic-Tac-Toe (Minimax)
4. 8-Puzzle (A^* Search)
5. Water Jug Problem
6. Travelling Salesman Problem
7. Tower of Hanoi
8. Monkey Banana Problem
9. Alpha-Beta Pruning
10. 8-Queens Problem

Experiment 1: Breadth First Search (BFS) using AIMA GRAPH-SEARCH

Aim

To implement Breadth-First Search using FIFO frontier as described in AIMA GRAPH-SEARCH algorithm.

Program:

```
from collections import deque

def bfs(graph, start, goal):
    frontier = deque([[start]])
    explored = set()
    while frontier:
        path = frontier.popleft()
        node = path[-1]
        print(f"Current Node: {node} | Path so far: {path}") # <-- Added
        if node == goal:
            return path
        if node not in explored:
            explored.add(node)
            for neighbor in graph[node]:
                frontier.append(path + [neighbor])
    return None

graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['G'], 'F': [], 'G': [] }
result = bfs(graph, 'A', 'G')
print("\nFinal BFS Path:", result)
```

Output

```
Output

Current Node: A | Path so far: ['A']
Current Node: B | Path so far: ['A', 'B']
Current Node: C | Path so far: ['A', 'C']
Current Node: D | Path so far: ['A', 'B', 'D']
Current Node: E | Path so far: ['A', 'B', 'E']
Current Node: F | Path so far: ['A', 'C', 'F']
Current Node: G | Path so far: ['A', 'B', 'E', 'G']

Final BFS Path: ['A', 'B', 'E', 'G']

=== Code Execution Successful ===
```

Viva Voce

1. Why BFS guarantees shortest path in unweighted graph?
2. What is space complexity of BFS?
3. Why is queue used in BFS?

Experiment 2: Depth First Search (DFS) using AIMA GRAPH-SEARCH

Aim

To implement Depth-First Search using LIFO frontier as per AIMA search algorithm.

Program

```
def dfs(graph, start, goal):  
    frontier = [[start]]  
    explored = set()  
    while frontier:  
        path = frontier.pop()  
        node = path[-1]  
        print(f"Current Node: {node} | Path so far: {path}") # <-- Added  
        if node == goal:  
            return path  
        if node not in explored:  
            explored.add(node)  
            for neighbor in graph[node]:  
                frontier.append(path + [neighbor])  
    return None  
  
graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['G'], 'F': [], 'G': [] }  
result = dfs(graph, 'A', 'G')  
print("\nFinal DFS Path:", result)
```

Output

```
Output

Current Node: A | Path so far: ['A']
Current Node: C | Path so far: ['A', 'C']
Current Node: F | Path so far: ['A', 'C', 'F']
Current Node: B | Path so far: ['A', 'B']
Current Node: E | Path so far: ['A', 'B', 'E']
Current Node: G | Path so far: ['A', 'B', 'E', 'G']

Final DFS Path: ['A', 'B', 'E', 'G']

=== Code Execution Successful ===
```

Viva Voce

1. Is DFS complete?
2. What is risk of infinite depth?
3. Compare BFS and DFS.

Experiment 3: Tic-Tac-Toe using Minimax using AIMA Adversarial Search

Aim

To implement Minimax algorithm for optimal decision making in adversarial environment.

Program

```
import math

def check_winner(board):

    wins = [(0,1,2),(3,4,5),(6,7,8),
            (0,3,6),(1,4,7),(2,5,8),
            (0,4,8),(2,4,6)]

    for a,b,c in wins:

        if board[a]==board[b]==board[c] and board[a]!=" ":

            return board[a]

    return None

def minimax(board, is_max):

    winner = check_winner(board)

    if winner == "X": return 1

    if winner == "O": return -1

    if " " not in board: return 0

    if is_max:

        best = -math.inf

        for i in range(9):

            if board[i]==" ":

                board[i]="X"

                best=max(best,minimax(board,False))
```

```
        board[i]=" "  
    return best  
else:  
    best = math.inf  
    for i in range(9):  
        if board[i]==" ":  
            board[i]="O"  
            best=min(best,minimax(board,True))  
            board[i]=" "  
    return best  
board=["X","O","X",  
        "O","X"," ",  
        " "," ","O"]  
print("Best value for X:", minimax(board, True))
```


Output

```
Output
Best value for X: 1

=== Code Execution Successful ===
```

Viva Voce

1. What is utility function?
2. What is game tree?
3. Why minimax ensures optimal play?

Experiment 4: 8-Puzzle using A* Search using AIMA Informed (Heuristic) Search

Aim

To solve 8-Puzzle using A* search with Manhattan distance heuristic.

Program

```
import heapq

goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan(state):
    distance = 0
    for i in range(1, 9):
        x1, y1 = divmod(state.index(i), 3)
        x2, y2 = divmod(goal.index(i), 3)
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

def print_board(state):
    for i in range(0, 9, 3):
        row = ""
        for j in range(3):
            val = state[i + j]
            row += " _ " if val == 0 else f" {val} "
        print(row)
    print()

def astar(start):
    frontier = []
```

```
heapq.heappush(frontier, (manhattan(start), 0, start, []))

explored = set()

while frontier:

    f, g, state, path = heapq.heappop(frontier)

    if state == goal:

        return path + [state]

    explored.add(state)

    zero = state.index(0)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    x, y = divmod(zero, 3)

    for dx, dy in moves:

        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:

            new = list(state)

            swap = nx * 3 + ny

            new[zero], new[swap] = new[swap], new[zero]

            new = tuple(new)

            if new not in explored:

                heapq.heappush(frontier, (g + 1 + manhattan(new), g + 1, new, path +
[state]))

    return None

start = (1, 2, 3, 4, 5, 6, 0, 7, 8)

solution = astar(start)

if solution:

    print(f"Solved in {len(solution) - 1} step(s)\n")

    for i, state in enumerate(solution):
```

```
if i == 0:

    print(f"--- Initial State ---")

else:

    print(f"--- Step {i} ---")

    print_board(state)

else:

    print("No solution found.")
```

Output

```
Output
Solved in 2 step(s)

--- Initial State ---
1  2  3
4  5  6
_  7  8

--- Step 1 ---
1  2  3
4  5  6
7  _  8

--- Step 2 ---
1  2  3
4  5  6
7  8  _

=== Code Execution Successful ===
```

Viva Voce

1. Why Manhattan is admissible?
2. Define $f(n)=g(n)+h(n)$.
3. Why A* is optimal?

Experiment 5: Water Jug Problem using BFS

Aim

To solve Water Jug problem using state-space search.

Program

```
from collections import deque

def water_jug(cap1, cap2, target):

    visited=set()

    queue=deque([((0,0),[])])

    while queue:

        (x,y),path=queue.popleft()

        if x==target or y==target:

            return path+[(x,y)]

        if (x,y) in visited:

            continue

        visited.add((x,y))

        next_states=[

            (cap1,y),(x,cap2),

            (0,y),(x,0),

            (min(cap1,x+y),y-(min(cap1,x+y)-x)),

            (x-(min(cap2,x+y)-y),min(cap2,x+y))

        ]

        for state in next_states:

            queue.append((state,path+[(x,y)]))

result=water_jug(4,3,2)

print("Solution Path:",result)
```

Output

Output

```
Solution Path: [(0, 0), (0, 3), (3, 0), (3, 3), (4, 2)]
```

```
=== Code Execution Successful ===
```

Viva Voce

1. What is state representation?
2. Why BFS suitable here?
3. Is solution optimal?

Experiment 6: Travelling Salesman using AIMA Uniform Cost Search Technique

Aim

To find minimum cost Hamiltonian cycle.

Program

```
import itertools

def tsp(graph,start):

    nodes=list(graph.keys())

    nodes.remove(start)

    min_cost=float('inf')

    best_path=None

    for perm in itertools.permutations(nodes):

        cost=0

        curr=start

        for node in perm:

            cost+=graph[curr][node]

            curr=node

        cost+=graph[curr][start]

        if cost<min_cost:

            min_cost=cost

            best_path=(start,)+perm+(start,)

    return best_path,min_cost

graph={ 'A':{'B':10,'C':15,'D':20}, 'B':{'A':10,'C':35,'D':25}, 'C':{'A':15,'B':35,'D':30},
        'D':{'A':20,'B':25,'C':30}    }

path,cost=tsp(graph,'A')

print("Best Path:",path, " and Total Cost is:", cost)
```


Output

```
Output
Best Path: ('A', 'B', 'D', 'C', 'A') and Total Cost is: 80
=== Code Execution Successful ===
```

Viva Voce

1. Why TSP is NP-Hard?
2. Difference between UCS and brute-force?
3. What is Hamiltonian cycle?

Experiment 7: Tower of Hanoi using AIMA Recursive Problem-Solving Approach

Aim

To solve Tower of Hanoi using recursive problem-solving.

Program

```
step = 0 # <-- Added: step counter

def hanoi(n, source, target, aux):

    global step

    if n == 1:

        step += 1

        print(f"Step {step}: Move disk 1 from {source} to {target}")

        return

    hanoi(n - 1, source, aux, target)

    step += 1

    print(f"Step {step}: Move disk {n} from {source} to {target}")

    hanoi(n - 1, aux, target, source)

print("=== Tower of Hanoi (3 disks) ===")

print("Pegs: A (Source) | B (Auxiliary) | C (Target)\n")

hanoi(3, 'A', 'C', 'B')

print(f"\nTotal steps: {step}")
```

Output

```
Output

=== Tower of Hanoi (3 disks) ===
Pegs: A (Source) | B (Auxiliary) | C (Target)

Step 1: Move disk 1 from A to C
Step 2: Move disk 2 from A to B
Step 3: Move disk 1 from C to B
Step 4: Move disk 3 from A to C
Step 5: Move disk 1 from B to A
Step 6: Move disk 2 from B to C
Step 7: Move disk 1 from A to C

Total steps: 7

=== Code Execution Successful ===
```

Viva Voce

1. What is recurrence relation?
2. Why complexity is $O(2^n)$?
3. Is this search problem?

Experiment 8: Monkey Banana Problem using AIMA (State-Space Search)

Aim

To implement simple goal-based problem solving.

Program

```
def monkey_banana():  
    state = {"monkey": "door", "box": "window", "banana": "middle", "has": False}  
    print("Initial State:", state)  
  
    # Step 1: Monkey walks from door to window  
    state["monkey"] = "window"  
    print("\nStep 1: Monkey walks from door to window")  
    print("State:", state)  
  
    # Step 2: Monkey pushes box from window to middle  
    state["box"] = "middle"  
    print("\nStep 2: Monkey pushes box from window to middle")  
    print("State:", state)  
  
    # Step 3: Monkey walks from window to middle  
    state["monkey"] = "middle"  
    print("\nStep 3: Monkey walks to middle (climbs box)")  
    print("State:", state)  
  
    # Step 4: Monkey grabs the banana  
    state["has"] = True  
    print("\nStep 4: Monkey grabs the banana!")  
    print("State:", state)  
  
    print("\n Goal Achieved: Monkey has the banana!")  
  
monkey_banana()
```

Output

```
Output

Initial State: {'monkey': 'door', 'box': 'window', 'banana': 'middle', 'has':
False}

Step 1: Monkey walks from door to window
State: {'monkey': 'window', 'box': 'window', 'banana': 'middle', 'has': False}

Step 2: Monkey pushes box from window to middle
State: {'monkey': 'window', 'box': 'middle', 'banana': 'middle', 'has': False}

Step 3: Monkey walks to middle (climbs box)
State: {'monkey': 'middle', 'box': 'middle', 'banana': 'middle', 'has': False}

Step 4: Monkey grabs the banana!
State: {'monkey': 'middle', 'box': 'middle', 'banana': 'middle', 'has': True}

Goal Achieved: Monkey has the banana!

=== Code Execution Successful ===
```

Viva Voce

1. Define state-space.
2. What is goal test?
3. What type of agent solves this?

Experiment 9: Alpha-Beta Pruning using AIMA Game Search Algorithm

Aim

To implement Alpha-Beta pruning for minimax optimization.

Program

```
import math

def alphabeta(node, depth, alpha, beta, maximizing):
    if depth == 0 or isinstance(node, int):
        return node
    if maximizing:
        value = -math.inf
        for child in node:
            value = max(value, alphabeta(child, depth - 1, alpha, beta, False))
            alpha = max(alpha, value)
            print(f"MAX | Value: {value} | Alpha: {alpha} | Beta: {beta}", end="")
            if alpha >= beta:
                print(" | Pruned!")
                break
        print()
        return value
    else:
        value = math.inf
        for child in node:
            value = min(value, alphabeta(child, depth - 1, alpha, beta, True))
            beta = min(beta, value)
            print(f"MIN | Value: {value} | Alpha: {alpha} | Beta: {beta}", end="")
            if beta <= alpha:
```

```
        print(" | Pruned!")
        break

    print()

    return value

tree = [[3, 5], [6, 9]]
print(f"Tree: {tree}\n")
result = alphabeta(tree, 2, -math.inf, math.inf, True)
print(f"\nOptimal Value: {result}")
```

Output

```
Output

Tree: [[3, 5], [6, 9]]

MIN | Value: 3 | Alpha: -inf | Beta: 3
MIN | Value: 3 | Alpha: -inf | Beta: 3
MAX | Value: 3 | Alpha: 3 | Beta: inf
MIN | Value: 6 | Alpha: 3 | Beta: 6
MIN | Value: 6 | Alpha: 3 | Beta: 6
MAX | Value: 6 | Alpha: 6 | Beta: inf

Optimal Value: 6

=== Code Execution Successful ===
```

Viva Voce

1. What is pruning?
2. Why $\alpha \leq \beta$?
3. Compare minimax and alpha-beta.

Experiment 10: 8-Queens Problem using AIMA Backtracking (CSP)

Aim

To solve 8-Queens using backtracking search.

Program

```
def solve_queens(n):  
    board=[-1]*n  
    def safe(row,col):  
        for i in range(row):  
            if board[i]==col or abs(board[i]-col)==row-i:  
                return False  
        return True  
    def backtrack(row):  
        if row==n:  
            print("Solution:",board)  
            return True  
        for col in range(n):  
            if safe(row,col):  
                board[row]=col  
                if backtrack(row+1):  
                    return True  
        return False  
    backtrack(0)  
    solve_queens(8)
```

Output

```
Output  
Solution: [0, 4, 7, 5, 2, 6, 1, 3]  
  
=== Code Execution Successful ===
```

Viva Voce

1. What is CSP?
2. What are constraints here?
3. What is branching factor?